Hearth & Table

Restaurant Ordering System

A coordinated Java application with six design patterns, a browser-based ordering experience, and a reproducible deployment.

TEAM MEMBER	ASSIGNED RESPONSIBILITY
Anuar	Builder + Factory Method
Akmaral	Strategy + Bridge
Mukhammad	Observer + Decorator

Java 21 / Spring Boot 3.5.16 / HTML, CSS & JavaScript

Report edition: 13 November 2026. Integrated baseline; contribution verification remains open until the other two members complete and merge their implementations.

Repository: github.com/ANAYSSA/Final-Project-SDP

Live demonstration: hearth-table.178.62.244.88.sslip.io

01 / INTRODUCTION

From separate exercises to one system

A restaurant order combines several independent decisions: what meal to create, which extras to add, how to assemble customer details, which pricing rule to apply, how to present a receipt and who should react to progress. Mixing these decisions in one controller makes changes difficult to explain and test.

Hearth & Table separates those decisions into six explicit design patterns. The customer can browse pizza, pasta and drinks; choose toppings; order for a table or takeaway; see a server-calculated quote; submit an order; and inspect status notifications and two kinds of receipt.

Project scope

This is an educational ordering demonstration. No payment is collected and no real kitchen, email or SMS service receives an order. A clearly labelled control advances the simulated kitchen status. The Observer mechanism executes in Java; the browser displays the resulting events.

Core requirements and evidence

REQUIREMENT	EVIDENCE
Six distinct patterns	Two creational, two structural and two behavioral patterns in the same flow.
Clean architecture	Six pattern packages with role-specific subfolders; Main is the sole root Java source.
Code + report	Runnable Java sources, tests, this PDF, UML sources/figures and real application screenshots.
Individual implementation	Assigned pairs are documented; Akmaral and Mukhammad must still implement and commit their contributions.
Submission	Every student submits a PDF report, source ZIP and GitHub link before the lecturer deadline. The supplied material did not state a date.

The stated grading weights are 50% code and Git link, 10% report, and 40% defense. The additional requirements specify a 5% deduction per violation. The checklist in docs/REQUIREMENTS.md is the submission control document.

02 / ARCHITECTURE

One application, six pattern packages

Main is the composition root and supports both Spring Boot startup and an integrated --demo console run. The builder package contains the order product and the application/HTTP boundary. All other domain packages express named pattern roles; static web assets live under src/main/resources/static.

PACKAGE	ROLE IN THE ORDER FLOW
factory	Menu definitions enter concrete creators and produce Meal implementations.
decorator	Selected extras wrap a Meal while preserving its component interface.
builder	Validated customer details and order lines become an immutable Order.
strategy	A selected discount algorithm computes the final BigDecimal price.
bridge	Receipt purpose and output format evolve independently.
observer	Status changes notify customer and kitchen subscribers.

Coordinated execution

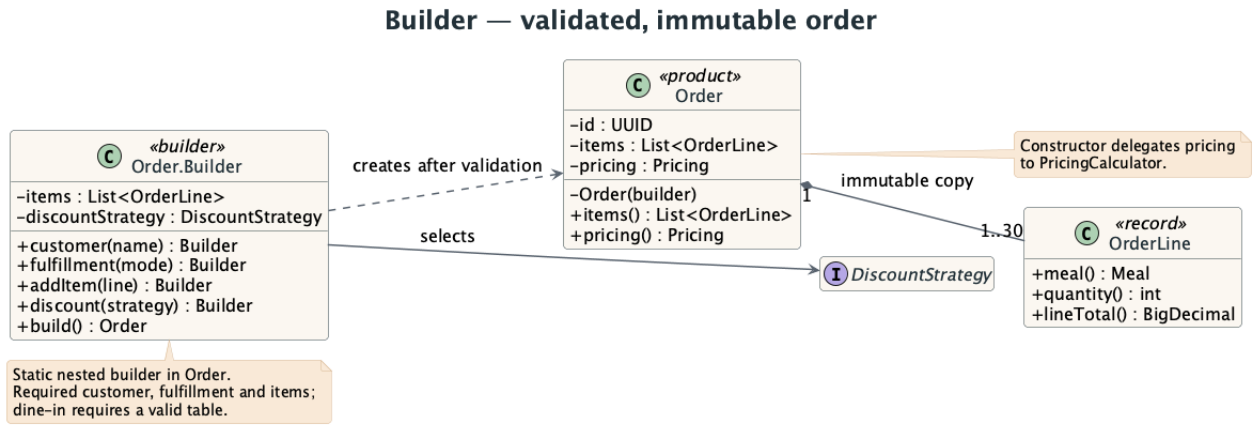
Factory Method creates a Meal; Decorator adds extras; Builder validates the order; Strategy prices it; Observer publishes progress; Bridge renders purpose-specific receipts. The landscape appendix shows the actual call sequence.

The browser sends identifiers and quantities, never authoritative prices. The service re-creates each meal and recalculates its decorated price. Receipt and history endpoints use the same session ownership checks.

Builder

Order.Builder accumulates customer, fulfillment, table, note, order lines and the chosen discount. build() validates the complete configuration before creating an immutable Order through a private constructor. The built product copies its item list, so later builder use cannot mutate an existing order.

A dine-in order requires a table from 1 to 30. Takeaway rejects a table. Blank names, empty orders and invalid quantities are rejected. Fluent construction keeps the call readable as optional details grow.



Java implementation excerpt

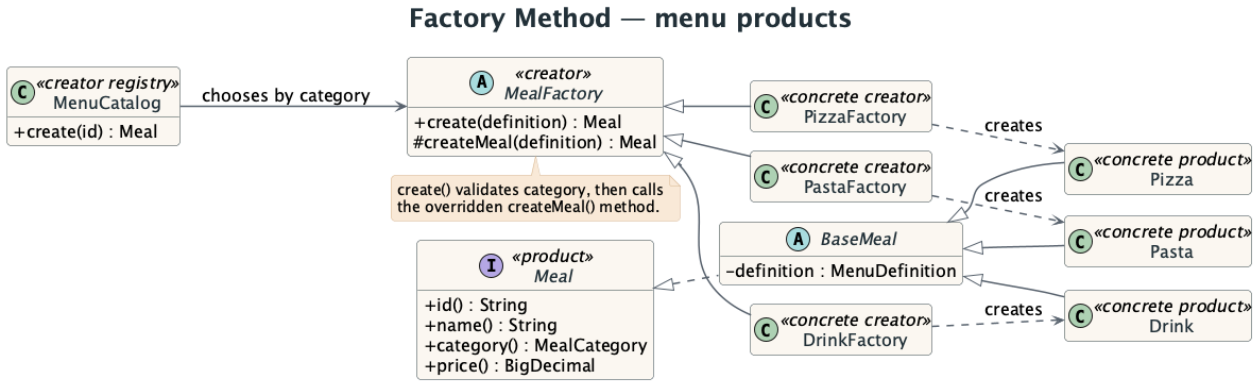
```
public static final class Builder {
    private String customerName;
    private Fulfillment fulfillment;
    private Integer tableNumber;
    private String note = "";
    private final List<OrderLine> items = new ArrayList<>();
    private DiscountStrategy discountStrategy = new NoDiscount();
    public Builder customer(String name) { customerName = name; return this; }
    public Builder fulfillment(Fulfillment mode) { fulfillment = mode; return this; }
    public Builder tableNumber(Integer table) { tableNumber = table; return this; }
    public Builder note(String value) { note = Objects.requireNonNull(value, "Note cannot be null");
     return this; }
    public Builder addItem(OrderLine line) { items.add(Objects.requireNonNull(line)); return this; }
    public Builder discount(DiscountStrategy strategy) { discountStrategy =
    Objects.requireNonNull(strategy); return this; }
    public Order build() {
        if (customerName == null || customerName.strip().length() < 2 ||
    customerName.strip().length() > 60)
            throw new IllegalArgumentException("Customer name must contain 2 to 60 characters");
        if (fulfillment == null) throw new IllegalArgumentException("Choose dine-in or takeaway");
```

src/main/java/com/restaurant/builder/product/Order.java

Factory Method

MealFactory is the abstract creator. Its public create operation validates the menu category and delegates construction to the protected factory method createMeal. PizzaFactory, PastaFactory and DrinkFactory override that method to create concrete Meal products.

MenuCatalog chooses a creator for the requested category. This registration step is not itself counted as the pattern: the abstraction and overriding creation operation are the Factory Method roles.



Java implementation excerpt

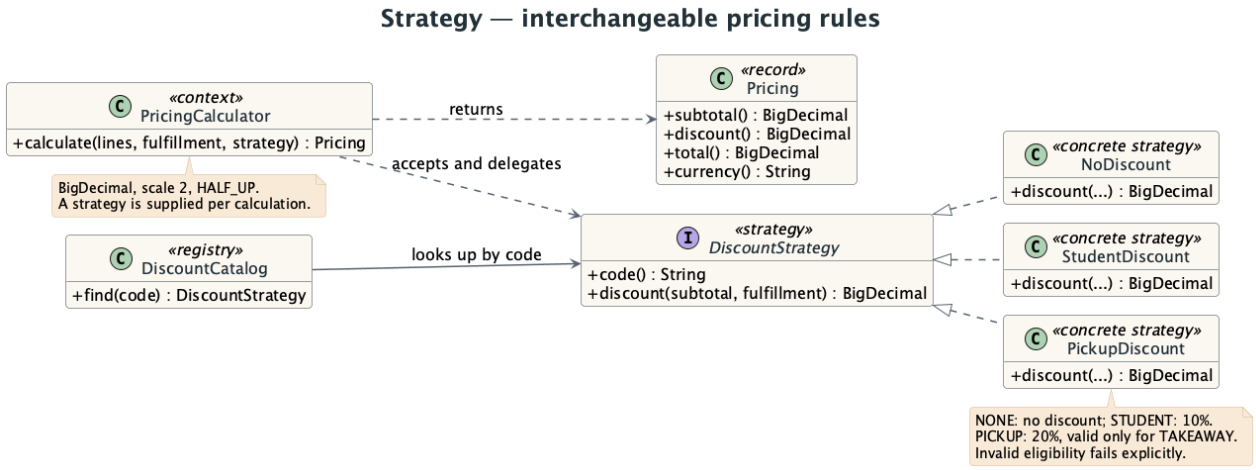
```
public abstract class MealFactory {
    private final MealCategory category;
    protected MealFactory(MealCategory category) { this.category = category; }
    public final Meal create(MenuDefinition definition) {
        Objects.requireNonNull(definition, "A menu definition is required");
        if (definition.category() != category) throw new IllegalArgumentException("Wrong meal
    category for factory");
        return createMeal(definition);
    }
    protected abstract Meal createMeal(MenuDefinition definition);
}
```

src/main/java/com/restaurant/factory/creator/MealFactory.java

Strategy

PricingCalculator is the context and depends on DiscountStrategy. NoDiscount, StudentDiscount and PickupDiscount implement interchangeable rules. Student discount is 10%; pickup discount is 20% and requires takeaway. Only one selected rule applies.

The decorated subtotal is calculated before discount. For two Margheritas with cheese: (2000 + 400) x 2 = 4800 KZT. Student discount is 480 KZT and the total is 4320 KZT. BigDecimal uses two decimal places and HALF_UP rounding.



Java implementation excerpt

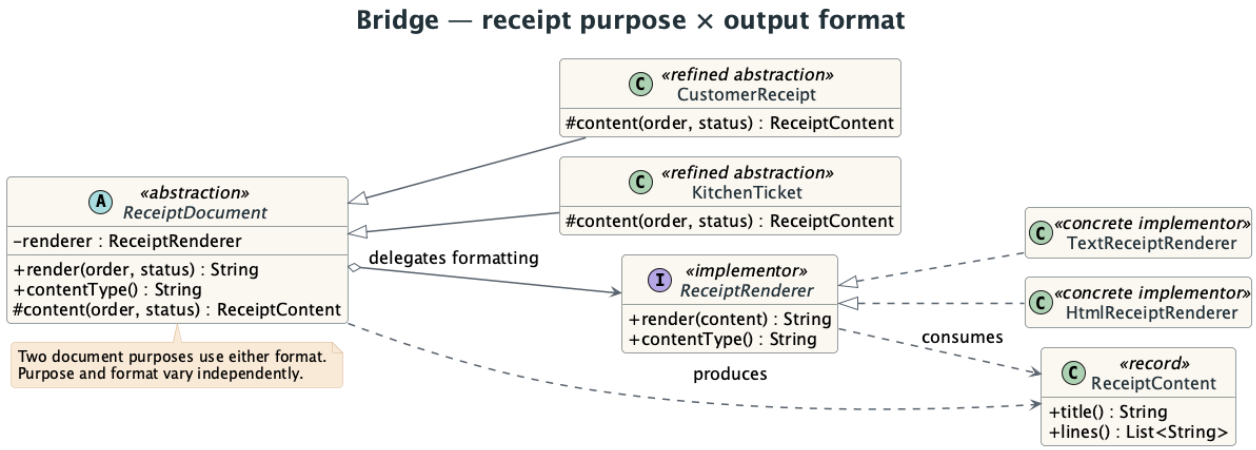
```
public final class PricingCalculator {
    public Pricing calculate(List<OrderLine> lines, Fulfillment fulfillment, DiscountStrategy
    strategy) {
        Objects.requireNonNull(fulfillment, "Choose dine-in or takeaway");
        Objects.requireNonNull(strategy, "A discount strategy is required");
        if (lines == null || lines.isEmpty() || lines.size() > 30) throw new
    IllegalArgumentException("An order must have 1 to 30 lines");
        BigDecimal subtotal = lines.stream().map(OrderLine::lineTotal).reduce(BigDecimal.ZERO,
    BigDecimal::add).setScale(2, RoundingMode.HALF_UP);
        BigDecimal discount = strategy.discount(subtotal, fulfillment).setScale(2,
    RoundingMode.HALF_UP);
        if (discount.signum() < 0 || discount.compareTo(subtotal) > 0) throw new
    IllegalArgumentException("Invalid discount amount");
        return new Pricing(subtotal, discount, subtotal.subtract(discount), "KZT");
    }
}
```

src/main/java/com/restaurant/strategy/context/PricingCalculator.java

Bridge

ReceiptDocument is the abstraction, refined by CustomerReceipt and KitchenTicket. ReceiptRenderer is the independent implementation hierarchy, with TextReceiptRenderer and HtmlReceiptRenderer. Each purpose can use either format through composition.

Customer receipts include prices and totals; kitchen tickets focus on preparation details. HTML output escapes dynamic content. A new renderer can serve both receipt types without creating a class for every purpose-format combination.



Java implementation excerpt

```
package com.restaurant.bridge.abstraction;

import com.restaurant.bridge.implementation.*;
import com.restaurant.builder.product.Order;
import com.restaurant.observer.event.OrderStatus;
import java.util.Objects;

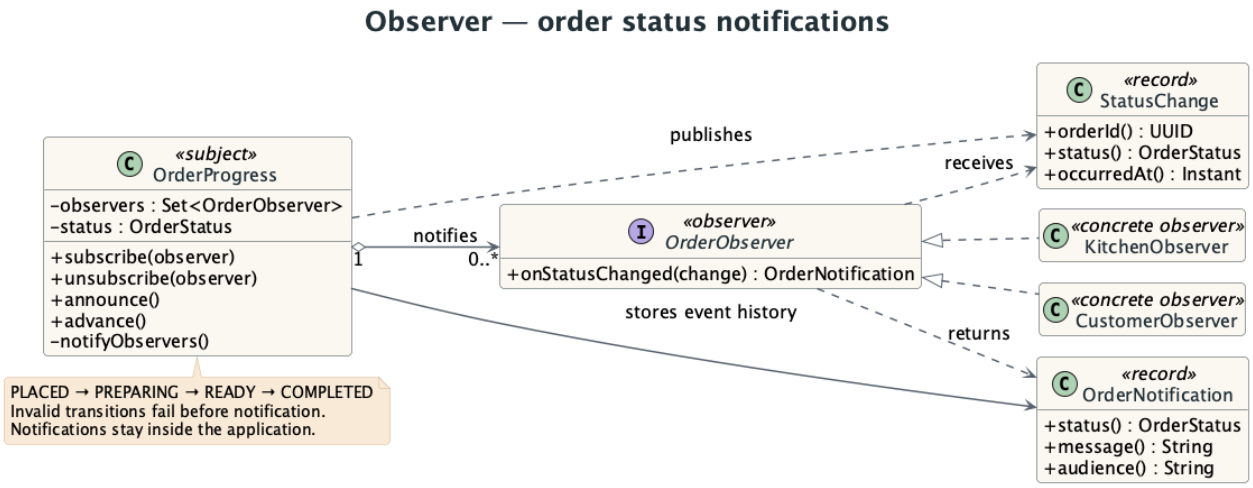
/** Abstraction: document purpose and output format can vary independently. */
public abstract class ReceiptDocument {
    private final ReceiptRenderer renderer;
    protected ReceiptDocument(ReceiptRenderer renderer) { this.renderer =
    Objects.requireNonNull(renderer); }
    public final String render(Order order, OrderStatus status) { return
    renderer.render(content(order, status)); }
    public final String contentType() { return renderer.contentType(); }
    protected abstract ReceiptContent content(Order order, OrderStatus status);
}
```

src/main/java/com/restaurant/bridge/abstraction/ReceiptDocument.java

Observer

OrderProgress is the subject. OrderObserver is the subscriber contract, implemented by CustomerObserver and KitchenObserver. It publishes the initial placement and valid changes; each subscriber produces an audience-specific notification.

The lifecycle is PLACED, PREPARING, READY, COMPLETED. Advancing a completed order is rejected. Subscription and unsubscription are explicit. Browser refresh is a display mechanism; notification dispatch is the backend Observer pattern.



Java implementation excerpt

```
package com.restaurant.observer.api;

import com.restaurant.observer.event.OrderNotification;
import com.restaurant.observer.event.StatusChange;

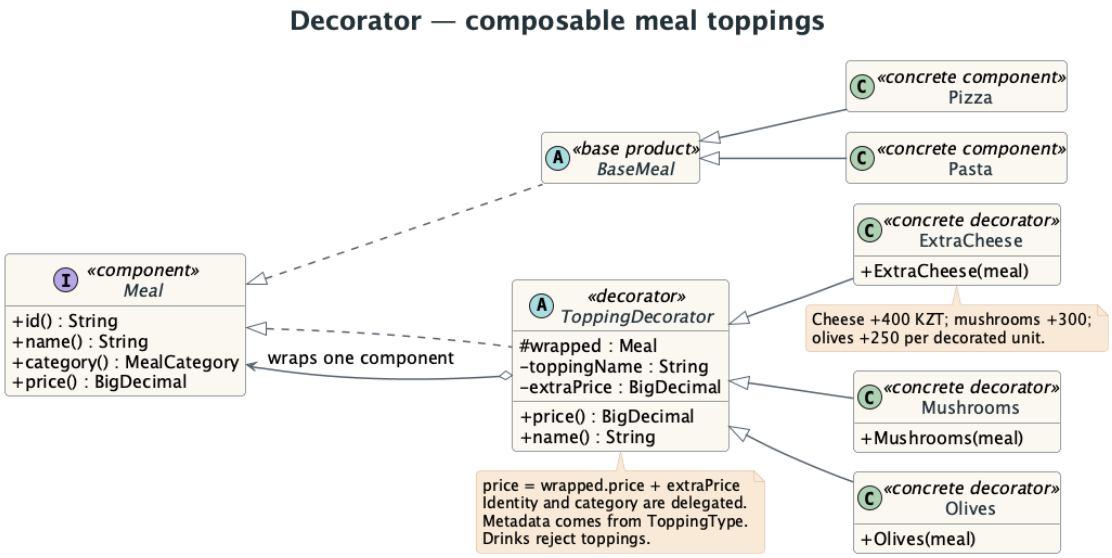
/** An in-app subscriber returns its notification; no external message is sent. */
public interface OrderObserver {
    OrderNotification onStatusChanged(StatusChange change);
}
```

src/main/java/com/restaurant/observer/api/OrderObserver.java

Decorator

Meal is the component shared by base products and wrappers. ToppingDecorator holds a wrapped Meal. ExtraCheese, Mushrooms and Olives add price and topping descriptions without modifying Pizza or Pasta classes.

Wrappers can compose in either order and preserve identity and category. Cheese adds 400 KZT, mushrooms 300 KZT and olives 250 KZT per unit. Drinks cannot receive toppings. The server applies each selected topping once and rejects duplicate IDs.



Java implementation excerpt

```
package com.restaurant.decorator.component;

import com.restaurant.factory.product.Meal;
import com.restaurant.factory.product.MealCategory;
import java.math.BigDecimal;
import java.util.Objects;

/** Decorators preserve the Meal contract and wrap another Meal, including a decorator. */
public abstract class ToppingDecorator implements Meal {
    protected final Meal wrapped;
    private final String toppingName;
    private final BigDecimal extraPrice;
    protected ToppingDecorator(Meal wrapped, ToppingType topping) {
        this.wrapped = Objects.requireNonNull(wrapped, "A meal is required");
        if (wrapped.category() == MealCategory.DRINK) throw new IllegalArgumentException("Toppings
    are available for pizza and pasta only");
        this.toppingName = topping.displayName();
        this.extraPrice = topping.price();
    }
    public final String id() { return wrapped.id(); }
    public final String name() { return wrapped.name() + " " + toppingName; }
    public final String description() { return wrapped.description() + ", " +
    toppingName.toLowerCase(); }
    public final MealCategory category() { return wrapped.category(); }
    public final BigDecimal price() { return wrapped.price().add(extraPrice); }
}
```

src/main/java/com/restaurant/decorator/component/ToppingDecorator.java

Menu and order composition

The menu combines category filters, search and item customization. Prices are in KZT. Toppings and discounts come from the backend catalog, so new registered implementations can appear without hard-coding their prices in the browser.

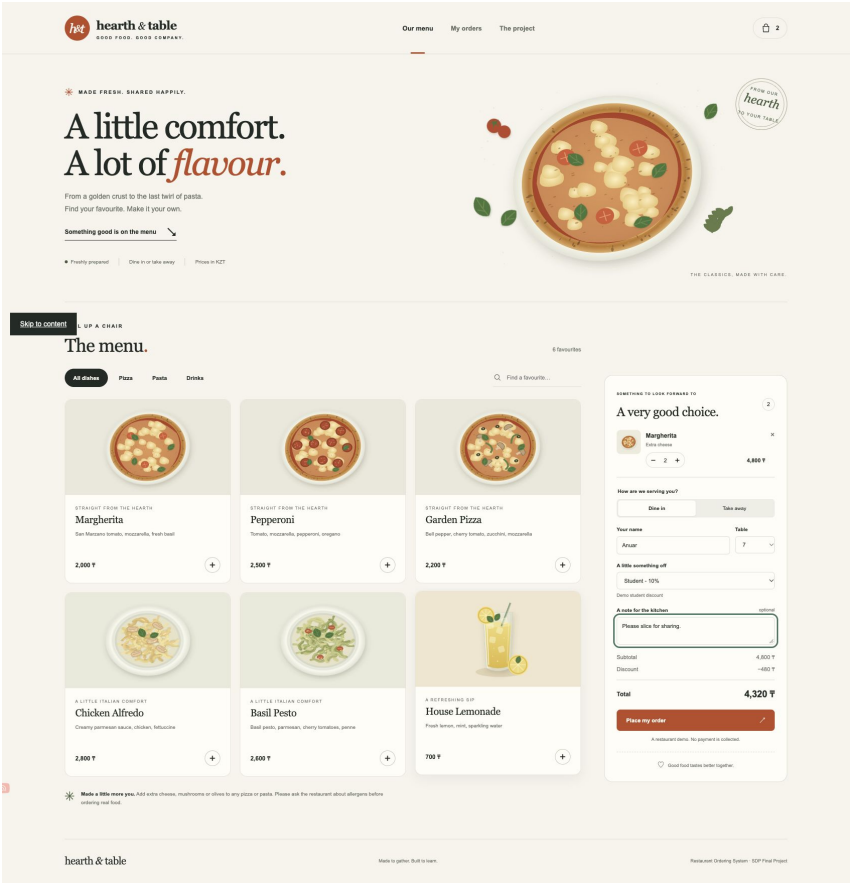


Figure 1. Real browser capture of the running application. The cart remains visible on wide screens; the layout adapts for narrower devices.

Checkout behavior

The browser requests a quote whenever the cart or pricing choices change. Older responses are ignored when a newer quote is pending. Order creation uses a UUID request identifier to make retries idempotent. Validation errors remain visible near the relevant workflow.

Customer details are used only by this demonstration. The API stores order history in the current HTTP session. The UI explains that a server restart or expired session clears this history.

Progress, notifications and receipts

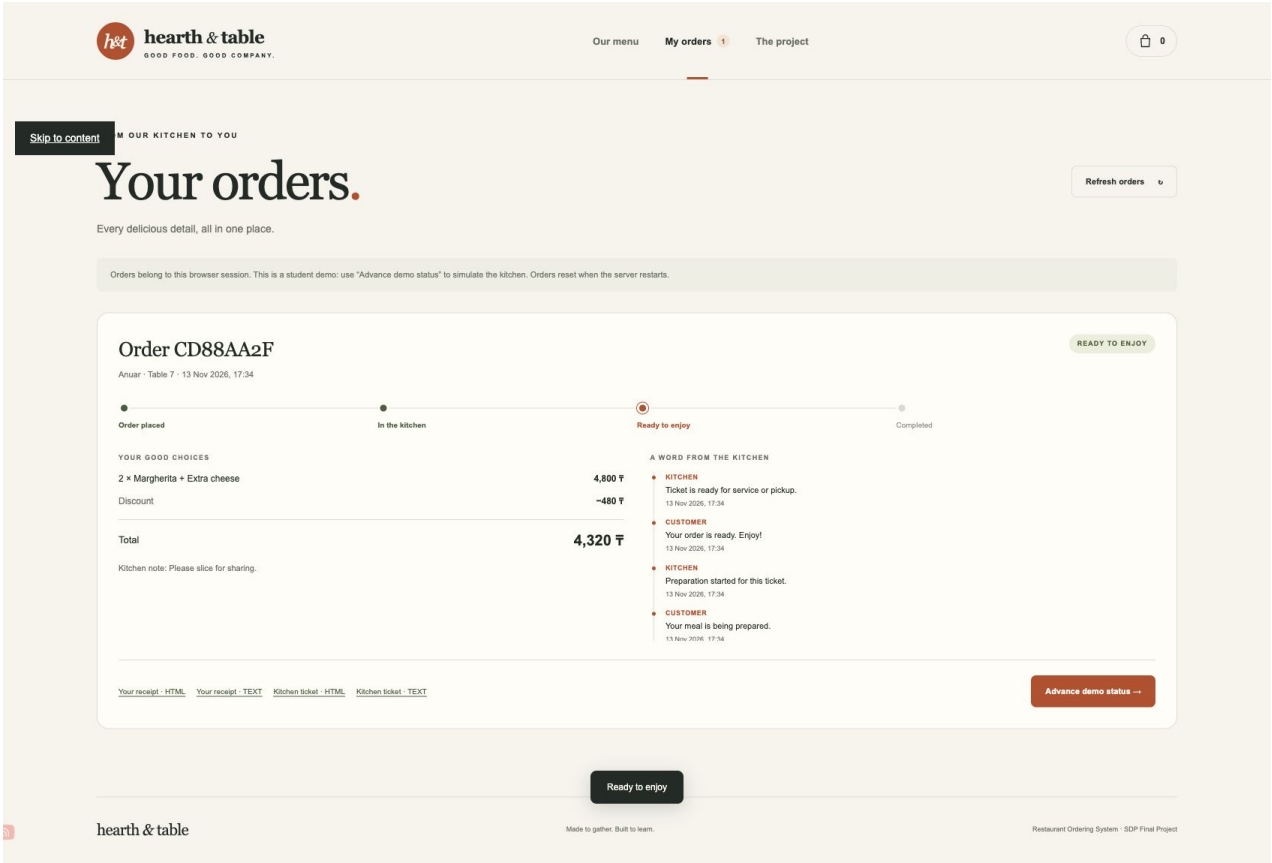


Figure 2. A placed order with its lifecycle and Java observer notifications. The demo status control makes simulation explicit.

An integrated demonstration

Create a Margherita with cheese, select a quantity of two and the student discount, then place the order. Inspect the total of 4320 KZT, advance the order status and show the new customer/kitchen events. Open both receipt types in text and HTML. This one sequence exercises all six patterns.

Independent verification

A second browser session cannot access the first session's order ID. Unsupported receipt formats, unknown menu IDs, toppings on drinks, invalid discounts and terminal state transitions return useful errors rather than arbitrary results.

Clean code backed by executable checks

PRINCIPLE	CONCRETE APPLICATION
Single responsibility	Factories create products; decorators add extras; pricing calculates totals; renderers format output.
Open/closed	New discount, topping, renderer and observer implementations extend existing contracts; registration exposes them.
Liskov substitution	Products and decorators satisfy Meal; each renderer serves both receipt abstractions.
Interface segregation	Meal, DiscountStrategy, ReceiptRenderer and OrderObserver expose narrow domain operations.
Dependency inversion	Pricing, receipt abstractions and the subject depend on interfaces rather than concrete variants.
KISS / DRY	One server calculation drives quote and placement; catalogs centralize prices; no database or payment integration is added without a requirement.

Automated verification

TEST SUITE	TESTS	FAILURES / ERRORS
OrderProgressTest	1	0
OrderApiTest	11	0
OrderBuilderTest	3	0
PricingTest	2	0
ConcurrentOrderTest	1	0
ReceiptBridgeTest	2	0
MealPatternTest	3	0

Verification commands: ./mvnw verify; python3 scripts/check_structure.py; java -jar target/hearth-and-table-1.0.0.jar --demo. GitHub Actions repeats these checks on push and pull request.

Tests target domain invariants and integration behavior: creation, wrapping, money, invalid input, request retries, session isolation, status transitions and escaped receipt content. Browser checks validate the customer workflow and responsive layout; these are separate from the Java test suite.

A reproducible teaching application

The application is deployed to the existing DigitalOcean Droplet. A Java 21 container runs the tested JAR; a separate Caddy container terminates HTTPS and proxies requests. The deployment uses a dedicated Compose project and caps memory consumption. No additional paid server is required.

Operational boundaries

This is a single-instance, session-based demonstration. Data is not durable across restarts, and the server shares resources with another service. Each session is bounded to 20 orders. There is no account system, real kitchen terminal, payment processor or external notification transport. These are deliberate limits of the implemented scope.

Conclusion

The result integrates six distinct patterns into a concrete customer journey. Creation, optional behavior, configuration, pricing, output and notifications have separate extension points. The same domain works through the browser and through the Main console demonstration, making pattern behavior inspectable during defense.

Further work

The next team iteration adds Akmaral's FAMILY strategy and JSON renderer, plus Mukhammad's herbs decorator and operations observer. Subsequent product work could add a durable database, staff authentication, audited kitchen controls, inventory availability and a payment-provider sandbox. Each addition should preserve the current domain contracts and session-ownership guarantees.

Before submitting

Complete both teammates' individual implementation work, preserve their original Git authors through PR merges, rerun the combined checks, update this report's contribution evidence and regenerate the source ZIP. Every student must submit personally by the lecturer's deadline. The repository alone does not satisfy individual implementation or submission requirements.

Project lineage and contribution record

The earlier assignments established a pizzeria domain and isolated pattern examples. The final project reuses the domain vocabulary and familiar pricing ideas while adding packages, a web API, validation, tests and an integrated customer flow.

Assignment 1 - Builder
https://github.com/ANAYSSA/SDPAssignment1Builder

Assignment 2 - Abstract Factory
https://github.com/ANAYSSA/Assignment2SDP_AbstractFactory_AnuarM

Assignment 3 - Bridge
https://github.com/ANAYSSA/sdp-assignment-3

Assignment 4 - Facade
https://github.com/ANAYSSA/as4-facade-pizzeria

Assignment 5 - Strategy
https://github.com/ANAYSSA/as5-strategy-pizzeria

Assignment 6 - Visitor
https://github.com/ANAYSSA/as6-visitor-pizzeria

Spring Boot system requirements
https://docs.spring.io/spring-boot/3.5/system-requirements.html

DigitalOcean Droplets documentation
https://docs.digitalocean.com/products/droplets/

Caddy automatic HTTPS
https://caddyserver.com/docs/automatic-https

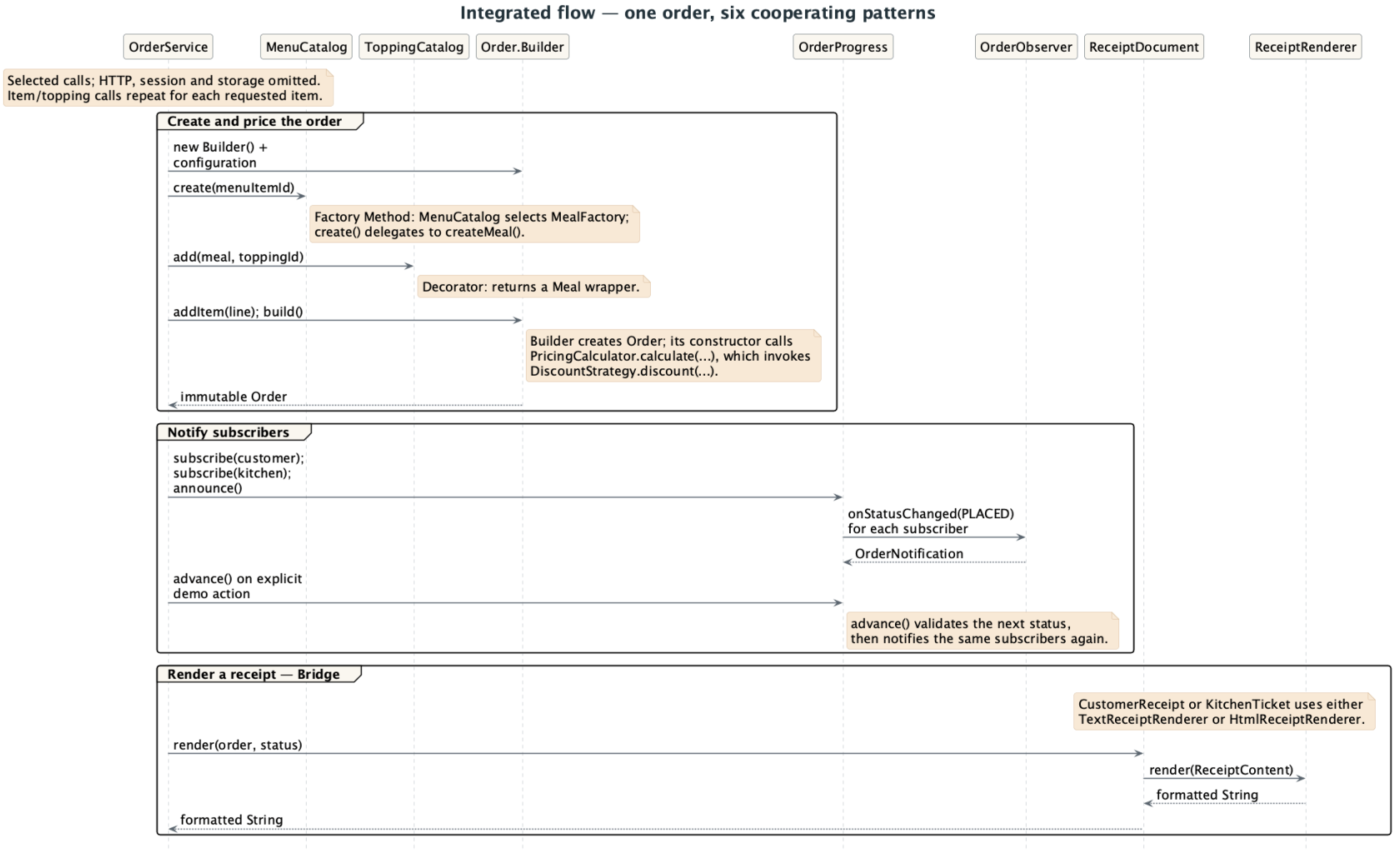
Authorship is evidenced by Git, not this table

MEMBER	BASELINE CONTRIBUTION STATUS
Anuar	Integrated baseline prepared and committed using the authenticated ANAYSSA identity; assigned Builder and Factory Method.
Akmaral	Assigned Strategy and Bridge. Individual extension commits and personal explanation pending.
Mukhammad	Assigned Observer and Decorator. Individual extension commits and personal explanation pending.

Inspect git log and actual diffs after merging the teammate PRs. docs/TEAM_TASKS.md contains their scoped implementation prompts and acceptance checks. Do not replace those contributions with fabricated author records. AI assistance does not remove the obligation to understand and defend the code.

Requirement sources: Final Project Defense Requirements.pdf supplied by the team, and the additional requirements supplied with the project request. No submission deadline was stated in those materials.

Integrated order sequence



Selected calls from the implemented application. The full editable PlantUML source and SVG export are in docs/uml.